LAB ACTIVITY MANUAL

Linux File System Exploration, Memory Acquisition,
and macOS Forensics

Digital Forensics Lab

1. Aim

To explore the Linux file system, analyze EXT4 metadata, perform memory acquisition using the LiME tool, collect system information using SysScout, and execute basic macOS forensic commands.

2. Objectives

- Understand the Linux file system structure.
- Identify mounted partitions and disk file system types.
- Analyze EXT4 file system metadata.
- Acquire a memory dump using LiME.
- Collect system information using SysScout.
- Perform basic macOS forensic commands.

3. Tools Required

Host System	macOS
Virtual Machine	UTM Virtual Machine
Guest OS	Ubuntu Linux
Version Control	Git
Memory Tool	LiME — Linux Memory Extractor

Recon Tool	SysScout
Terminals	Linux Terminal & macOS Terminal

---

4. Theory Overview

4.1 Linux File System

The Linux file system organizes data into directories and files. Modern Linux distributions commonly use the EXT4 file system, which provides journaling, improved performance, and better reliability.

4.2 Memory Forensics

Memory forensics involves capturing and analyzing the contents of volatile memory (RAM) to detect running processes, network connections, malware artifacts, and other forensic evidence.

4.3 LiME (Linux Memory Extractor)

LiME is a loadable kernel module (LKM) used to acquire a complete memory dump from a running Linux system. The dump can later be analyzed using tools such as Volatility.

4.4 SysScout

SysScout is a shell-based tool that collects system information including running processes, hardware details, and forensic artifacts from a live Linux system.

---

5. Prerequisites — Install Required Packages

Before starting the lab, ensure the following packages are installed on your Ubuntu VM. Open a terminal and run the commands below.

5.1 Update Package List

```
sudo apt update
```

5.2 Install Git

Git is required to clone the LiME and SysScout repositories.

```
sudo apt install git -y
```

5.3 Install Make and GCC (Build Tools)

Make and GCC are needed to compile the LiME kernel module from source.

```
sudo apt install build-essential -y
```

This installs: gcc, g++, make, and other essential build tools in one command.

5.4 Install Linux Kernel Headers

LiME compiles against your running kernel. You must install the matching kernel header files. First, check your exact kernel version:

```
uname -r
```

This prints your kernel version string (e.g., `6.8.0-49-generic`). Use that string in the next command:

```
sudo apt install linux-headers-$(uname -r) -y
```

> ***Note:*** *The $(uname -r) part automatically substitutes your kernel version. This also determines the exact .ko filename that LiME produces after compilation.*

5.5 How to Find the Correct .ko Filename

After compiling LiME, the .ko file is named after your kernel version. Run the following to see the exact filename:

```
ls LiME/src/*.ko
```

The output will look like:

```
lime-6.8.0-49-generic.ko
```

Use that exact filename in the insmod command in Step 2.4.

5.6 Install sqlite3 (for macOS History — optional on Ubuntu)

On macOS, sqlite3 is pre-installed. On Ubuntu (if needed for testing):

```
sudo apt install sqlite3 -y
```

---

6. Pre-Lab Setup — Installing UTM & Ubuntu

Before starting the lab, you need a working Ubuntu Linux environment. This section walks you through installing Ubuntu on macOS using UTM, and on Windows using VirtualBox — step by step.

Section A: macOS — Install UTM

UTM is a free virtual machine app for macOS that lets you run Linux and other operating systems on your Mac.

Step A.1 — Download UTM

Open Safari or any browser and go to:

```
https://mac.getutm.app
```

Click the Download button. This will download the UTM .dmg installer file to your Downloads folder.

Step A.2 — Install UTM

- Open the downloaded .dmg file from your Downloads folder.
- A window appears showing the UTM icon and an Applications folder shortcut.
- Drag the UTM icon into the Applications folder.
- Once copying finishes, eject the .dmg and open UTM from Applications or Launchpad.
- If macOS shows a warning saying the app is from an unidentified developer, go to: System Settings → Privacy & Security → scroll down → click Open Anyway.

Section B: macOS — Install Ubuntu on UTM

Now that UTM is installed, follow these steps to create an Ubuntu virtual machine.

Step B.1 — Download Ubuntu ISO

Go to the official Ubuntu website and download the Ubuntu 22.04 LTS desktop ISO:

```
https://ubuntu.com/download/desktop
```

Download the file named: ubuntu-22.04.x-desktop-amd64.iso (approximately 4.7 GB). Save it to your Downloads folder.

> ***Note:*** *For Apple Silicon Macs (M1/M2/M3), download the ARM version instead: ubuntu-22.04.x-desktop-arm64.iso from https://ubuntu.com/download/server/arm*

Step B.2 — Create a New Virtual Machine in UTM

- Open UTM from your Applications folder.
- Click the + button (Create a New Virtual Machine).
- Select Virtualize (for Apple Silicon Macs) or Emulate (for Intel Macs running ARM images).
- Under Operating System, select Linux.
- Click Browse and select the Ubuntu ISO file you downloaded in Step B.1.
- Click Continue.

Step B.3 — Configure Hardware

- Memory (RAM): Set to at least 2048 MB (2 GB). Recommended: 4096 MB (4 GB).
- CPU Cores: Set to at least 2 cores.
- Click Continue.

Step B.4 — Configure Storage

- Storage Size: Set to at least 20 GB. Recommended: 30 GB.
- Click Continue.
- On the Summary page, give your VM a name (e.g., Ubuntu-Lab).
- Click Save.

Step B.5 — Start the VM and Install Ubuntu

- In UTM, click the Play button next to your Ubuntu-Lab VM.
- The VM will boot into the Ubuntu Live installer.
- Select your language and click "Install Ubuntu".
- Choose "Normal Installation". Check both boxes for downloading updates and third-party software.
- Select "Erase disk and install Ubuntu" (this only erases the virtual disk, not your Mac).
- Set your name, computer name, username, and password. Remember this password — you will need it for sudo commands.
- Click "Install Now" and wait for installation to complete (10–20 minutes).
- When prompted, click "Restart Now". Press Enter when asked to remove the installation medium.
- Ubuntu will reboot and you will be taken to the login screen.

> ***Note:*** *After Ubuntu starts, open a terminal (Ctrl+Alt+T) and run: sudo apt update && sudo apt upgrade -y to ensure all packages are up to date before starting the lab.*

Section C: Windows — Install Ubuntu using VirtualBox

Windows users will use VirtualBox — a free, open-source virtual machine application from Oracle — to run Ubuntu Linux.

Step C.1 — Download VirtualBox

Go to the official VirtualBox website:

```
https://www.virtualbox.org/wiki/Downloads
```

Click on Windows hosts to download the VirtualBox installer (.exe file, approximately 100 MB).

Step C.2 — Install VirtualBox

- Double-click the downloaded .exe installer.
- Click Next through the setup wizard, keeping all default options.
- When asked about network interfaces, click Yes to allow the installation.
- Click Install. If Windows asks for permission (UAC prompt), click Yes.
- Click Finish once installation completes. VirtualBox will open automatically.

> ***Note:*** *If you see a warning about missing dependencies (e.g., Visual C++ Redistributable), download and install it from Microsoft's website, then re-run the VirtualBox installer.*

Step C.3 — Download Ubuntu ISO

Go to the official Ubuntu website:

```
https://ubuntu.com/download/desktop
```

Download ubuntu-22.04.x-desktop-amd64.iso (approximately 4.7 GB) and save it to your Downloads folder.

Step C.4 — Create a New Virtual Machine in VirtualBox

- Open VirtualBox. Click New (the blue star icon at the top).
- Name: Type Ubuntu-Lab.
- Folder: Leave as default.
- ISO Image: Click the dropdown, select Other, and browse to your downloaded Ubuntu ISO file.
- Check the box "Skip Unattended Installation" so you can set it up manually.
- Click Next.

Step C.5 — Configure Hardware

- Base Memory: Set to at least 2048 MB (2 GB). Recommended: 4096 MB.
- Processors: Set to at least 2.
- Click Next.

Step C.6 — Configure Virtual Hard Disk

- Select "Create a Virtual Hard Disk Now".
- Disk Size: Set to at least 25 GB. Recommended: 30 GB.
- Click Next, then click Finish.

Step C.7 — Start the VM and Install Ubuntu

- Select Ubuntu-Lab in the left panel and click Start (the green arrow).
- The VM boots into the Ubuntu Live installer.
- Click "Try or Install Ubuntu", then select your language and click "Install Ubuntu".
- Choose "Normal Installation". Check both update options.
- "Erase disk and install Ubuntu" — this only affects the virtual disk, not your Windows files.
- Set your name, username, and password. Remember this for sudo commands.
- Click "Install Now" → Continue → wait 10–20 minutes.
- When installation finishes, click Restart Now and press Enter when prompted.
- Ubuntu boots to the login screen. Enter your password to log in.

> ***Note:*** *After logging in, open a terminal (Ctrl+Alt+T) and run: sudo apt update && sudo apt upgrade -y before starting the lab.*

Step C.8 — Install VirtualBox Guest Additions (Recommended)

Guest Additions improve the VM experience — enables full-screen mode, shared clipboard, and better display resolution.

- With Ubuntu running in VirtualBox, click the Devices menu at the top of the VirtualBox window.
- Click "Insert Guest Additions CD Image..."
- Ubuntu will prompt you to run the software. Click Run.
- Enter your password when asked. Wait for installation to finish.
- Restart the VM. You can now resize the window and use full screen (View → Full Screen).

Section D: Quick Comparison — UTM vs VirtualBox

Feature	UTM (macOS)	VirtualBox (Windows)
Platform	macOS only	Windows, macOS, Linux
Cost	Free	Free
Apple Silicon	Full support (M1/M2/M3)	Limited support
Performance	Excellent on Mac	Good on Windows
Guest Additions	SPICE tools (optional)	VBox Guest Additions (recommended)
Recommended RAM	4 GB for Ubuntu	4 GB for Ubuntu

Recommended Disk	30 GB	30 GB

7. Procedure

Part 1: Exploring the Linux File System

Step 1.1 — View All Mounted Partitions

Run the following command to display all mounted partitions and their disk usage:

```
df -h
```

```
prajwal@prajwal-QEMU-Virtual-Machine:~/Desktop$ df -h
Filesystem      Size  Used Avail Use% Mounted on
tmpfs           677M  2.0M  675M   1% /run
/dev/vda1        24G   15G  8.1G  64% /
tmpfs           1.7G     0  1.7G   0% /dev/shm
efivarfs        256K   14K  243K   6% /sys/firmware/efi/efivars
tmpfs           5.0M  8.0K  5.0M   1% /run/lock
tmpfs           1.0M     0  1.0M   0% /run/credentials/systemd-journald.service
tmpfs           1.0M     0  1.0M   0% /run/credentials/systemd-resolved.service
/dev/vda2       1.1G  6.6M  1.1G   1% /boot/efi
tmpfs           1.7G  8.0K  1.7G   1% /tmp
tmpfs           1.0M     0  1.0M   0% /run/credentials/serial-getty@ttyAMA0.service
tmpfs           339M  132K  339M   1% /run/user/1000
prajwal@prajwal-QEMU-Virtual-Machine:~/Desktop$
```

Output of df -h showing mounted partitions

What to notice:

- **Filesystem column:** shows the device name (e.g., /dev/sda1).
- **Mounted on column:** shows where the partition is accessible.
- **/ (root partition):** where EXT4 is typically used.

Step 1.2 — View Disk and File System Types

List all block devices with their file system types and UUIDs:

```
lsblk -f
```

```
prajwal@prajwal-QEMU-Virtual-Machine:~/Desktop$ lsblk -f
NAME   FSTYPE   FSVER LABEL UUID                                 FSAVAIL FSUSE% MOUNTPOINTS
loop0  squashfs 4.0                                                    0   100% /snap/bare/5
loop1  squashfs 4.0                                                    0   100% /snap/core22/2134
loop2  squashfs 4.0                                                    0   100% /snap/desktop-security-center/60
loop3  squashfs 4.0                                                    0   100% /snap/firefox/6961
loop4  squashfs 4.0                                                    0   100% /snap/gnome-42-2204/228
loop5  squashfs 4.0                                                    0   100% /snap/prompting-client/105
loop6  squashfs 4.0                                                    0   100% /snap/gtk-common-themes/1535
loop7  squashfs 4.0                                                    0   100% /snap/snap-store/1301
loop8  squashfs 4.0                                                    0   100% /snap/snapd/25205
loop9  squashfs 4.0                                                    0   100% /snap/snapd-desktop-integration/316
vda
├─vda1 ext4     1.0         73fad143-addd-4f41-8c3c-2c7b009ac553      8G    61% /
└─vda2 vfat     FAT32       E078-2146                                 1G     1% /boot/efi
prajwal@prajwal-QEMU-Virtual-Machine:~/Desktop$
```

Output of lsblk -f showing disk and filesystem types

What to notice:

- **FSTYPE:** indicates the file system type (EXT4).
- **UUID:** each partition has a unique identifier.
- **Forensic use:** UUIDs help uniquely identify evidence partitions.

Step 1.3 — Read the EXT4 Superblock

Read the EXT4 superblock metadata to understand the file system structure:

```
sudo tune2fs -l /dev/vda1
```

```
prajwal@prajwal-QEMU-Virtual-Machine:~/Desktop$ sudo tune2fs -l /dev/vda1
tune2fs 1.47.2 (1-Jan-2025)
Filesystem volume name:   <none>
Last mounted on:          /
Filesystem UUID:          73fad143-addd-4f41-8c3c-2c7b009ac553
Filesystem magic number:  0xEF53
Filesystem revision #:    1 (dynamic)
Filesystem features:      has_journal ext_attr resize_inode dir_index orphan_file filetype needs_recovery extent 64bit flex_bg metadata_csum_seed sparse_super large_file huge
_file dir_nlink extra_isize metadata_csum orphan_present
Filesystem flags:         unsigned_directory_hash
Default mount options:    user_xattr acl
Filesystem state:         clean
Errors behavior:          Continue
Filesystem OS type:       Linux
Inode count:              1569792
Block count:              6277888
Reserved block count:     313894
Overhead clusters:        142572
Free blocks:              3546780
Free inodes:              1359248
First block:              0
Block size:               4096
Fragment size:            4096
Group descriptor size:    64
```

tune2fs output showing EXT4 superblock information

What to notice:

- **Magic number (0xEF53):** confirms this is an EXT4 partition.
- **Block size:** data stored in 4096-byte blocks.
- **Last mount time:** shows when the disk was last accessed.
- **Inode count:** total files/directories the filesystem supports.

Part 2: Memory Acquisition Using LiME

> ***Note:*** *Make sure you have completed Section 5 (Prerequisites) before starting this part.*

Step 2.1 — Download LiME

Clone the LiME repository from GitHub:

```
git clone https://github.com/504ensicsLabs/LiME.git
```

```
prajwal@prajwal-QEMU-Virtual-Machine:~/Desktop/LiME/src$ git clone https://github.com/504ensicsLabs/LiME.git
Cloning into 'LiME'...
remote: Enumerating objects: 418, done.
remote: Counting objects: 100% (39/39), done.
remote: Compressing objects: 100% (26/26), done.
remote: Total 418 (delta 21), reused 15 (delta 12), pack-reused 379 (from 3)
Receiving objects: 100% (418/418), 1.63 MiB | 2.05 MiB/s, done.
Resolving deltas: 100% (219/219), done.
prajwal@prajwal-QEMU-Virtual-Machine:~/Desktop/LiME/src$
```

Git clone output for LiME repository

Step 2.2 — Compile LiME

Navigate to the source directory and build the kernel module:

```
cd LiME/src

make clean

ls

make
```

```
prajwal@prajwal-QEMU-Virtual-Machine:~/Desktop/LiME$ cd ..
prajwal@prajwal-QEMU-Virtual-Machine:~/Desktop$ cd LiME/
prajwal@prajwal-QEMU-Virtual-Machine:~/Desktop/LiME$ cd src
prajwal@prajwal-QEMU-Virtual-Machine:~/Desktop/LiME/src$ make clean
rm -f *.o *.mod.c Module.symvers Module.markers modules.order \.*.o.cmd \.*.ko.cmd \.*.o.d
rm -rf \.tmp_versions
prajwal@prajwal-QEMU-Virtual-Machine:~/Desktop/LiME/src$ make clean
rm -f *.o *.mod.c Module.symvers Module.markers modules.order \.*.o.cmd \.*.ko.cmd \.*.o.d
rm -rf \.tmp_versions
prajwal@prajwal-QEMU-Virtual-Machine:~/Desktop/LiME/src$ ls
LiME       Makefile.sample  disk.c  lime-6.17.0-19-generic.ko  lime.mod  tcp.c
Makefile   deflate.c        hash.c  lime.h                     main.c
prajwal@prajwal-QEMU-Virtual-Machine:~/Desktop/LiME/src$ make
make -C /lib/modules/6.17.0-19-generic/build M="/home/prajwal/Desktop/LiME/src" modules
make[1]: Entering directory '/usr/src/linux-headers-6.17.0-19-generic'
make[2]: Entering directory '/home/prajwal/Desktop/LiME/src'
warning: the compiler differs from the one used to build the kernel
  The kernel was built by: aarch64-linux-gnu-gcc (Ubuntu 15.2.0-4ubuntu4) 15.2.0
  You are using:           gcc (Ubuntu 15.2.0-4ubuntu4) 15.2.0
  CC [M]  tcp.o
  CC [M]  disk.o
  CC [M]  main.o
  CC [M]  hash.o
  CC [M]  deflate.o
  LD [M]  lime.o
  MODPOST Module.symvers
WARNING: modpost: missing MODULE_DESCRIPTION() in lime.o
  CC [M]  lime.mod.o
  CC [M]  .module-common.o
  LD [M]  lime.ko
  BTF [M] lime.ko
Skipping BTF generation for lime.ko due to unavailability of vmlinux
make[2]: Leaving directory '/home/prajwal/Desktop/LiME/src'
make[1]: Leaving directory '/usr/src/linux-headers-6.17.0-19-generic'
strip --strip-unneeded lime.ko
mv lime.ko lime-6.17.0-19-generic.ko
prajwal@prajwal-QEMU-Virtual-Machine:~/Desktop/LiME/src$
```

LiME compilation output using make

After successful compilation, a `.ko` file will appear in the directory listing.

Step 2.3 — Find Your Kernel Version (for .ko filename)

Before loading the module, check your kernel version to get the exact .ko filename:

```
uname -r
```

Then list the compiled .ko file to confirm the full name:

```
ls LiME/src/*.ko
```

> ***Note:*** *The .ko filename must exactly match your running kernel version. Example: lime-6.8.0-49-generic.ko — replace this with your actual output.*

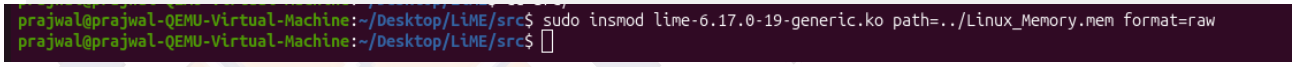
Step 2.4 — Acquire Memory Dump

Insert the kernel module to acquire a raw memory dump. Replace the filename below with your actual .ko filename from the previous step:

```
sudo insmod lime-$(uname -r).ko path=../Linux_Memory.mem format=raw
```

Or using the explicit filename:

```
sudo insmod lime-6.8.0-49-generic.ko path=../Linux_Memory.mem format=raw
```



insmod command loading LiME kernel module

This loads LiME into the kernel and writes the full memory image in raw format.

Step 2.5 — Verify Memory Dump

Confirm the memory dump file was created successfully:

```
ls -lh ../Linux_Memory.mem
```



Verification showing the memory dump file

The file size should match the total RAM installed on the system.

Part 3: System Information Collection Using SysScout

Step 3.1 — Download SysScout

Navigate to the Desktop and clone the SysScout repository:

```
cd ~/Desktop

git clone https://github.com/joshbrunty/SysScout.git
```

```
prajwal@prajwal-QEMU-Virtual-Machine:~$ cd ~/Desktop
prajwal@prajwal-QEMU-Virtual-Machine:~/Desktop$ git clone https://github.com/joshbrunty/SysScout.git
Cloning into 'SysScout'...
remote: Enumerating objects: 108, done.
remote: Total 108 (delta 0), reused 0 (delta 0), pack-reused 108 (from 1)
Receiving objects: 100% (108/108), 27.11 KiB | 660.00 KiB/s, done.
Resolving deltas: 100% (50/50), done.
prajwal@prajwal-QEMU-Virtual-Machine:~/Desktop$ ls
LiME  SysScout  github.com
prajwal@prajwal-QEMU-Virtual-Machine:~/Desktop$ cd SysScout/
prajwal@prajwal-QEMU-Virtual-Machine:~/Desktop/SysScout$
```

Git clone output for SysScout

Step 3.2 — Run SysScout

Navigate into the SysScout directory, grant execute permission, and run:

```
cd SysScout

chmod +x SysScout.sh

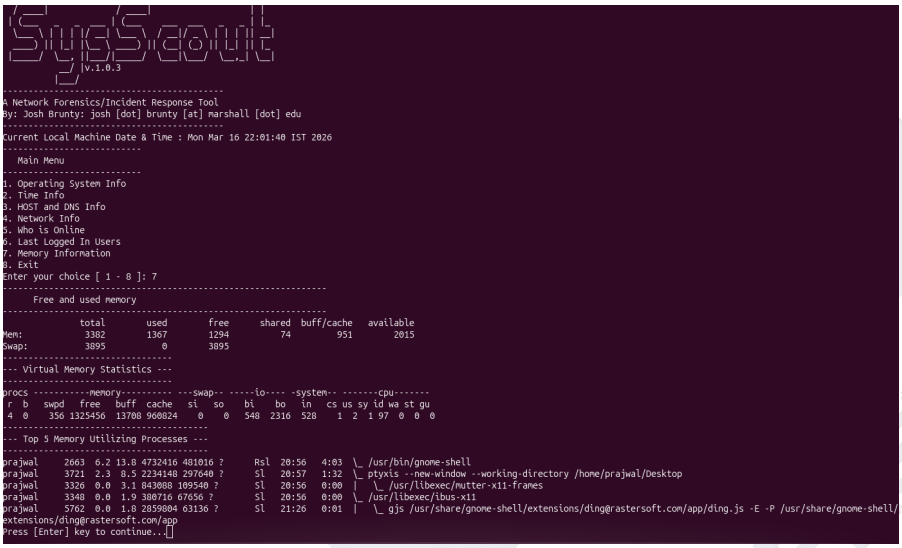
./SysScout.sh
```

SysScout displays an interactive menu. Try the following options:

- **Option 1:** Collect basic system information (hostname, OS, uptime).
- **Option 7:** Gather extended forensic artifacts.



SysScout menu — Option 1 output



SysScout menu — Option 7 output

Part 4: Understanding Files and Inodes (Linux & macOS)

In Linux and macOS, everything is treated as a file. This section explores different file types and inode metadata, which are crucial in digital forensic analysis.

4.0 — Device Naming Conventions: sda vs vda vs disk0

Before running commands, it is important to understand why disk device names look different across systems. The device name depends on the type of machine and OS you are using.

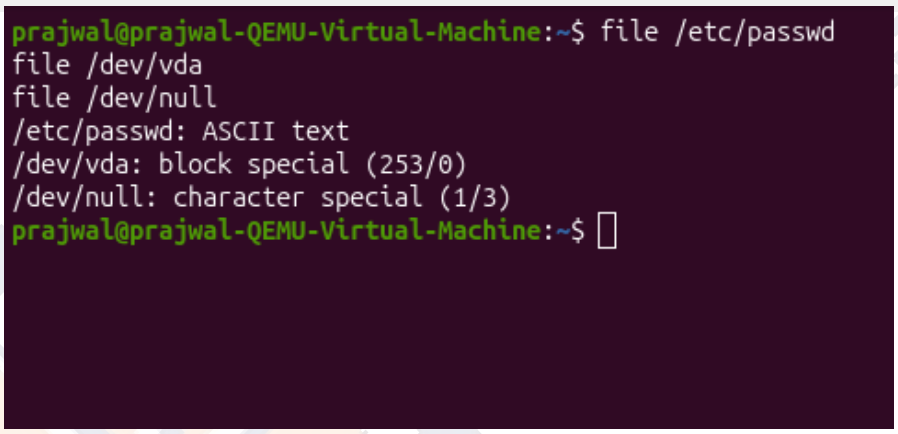
Device Name	Where you see it	What it means
sda, sdb, sdc...	Linux (physical or SATA disk)	SCSI/SATA disk. sda = 1st disk, sdb = 2nd disk, sda1 = 1st partition
vda, vdb...	Linux inside a Virtual Machine (UTM, KVM, QEMU)	Virtual disk. vda = 1st virtual disk, vda1 = 1st partition on it
nvme0n1...	Linux with NVMe SSD	NVMe storage. nvme0n1 = 1st drive, nvme0n1p1 = 1st partition
disk0, disk1...	macOS (all disk types)	macOS names all disks as disk0, disk1. disk0s1 = 1st partition of disk0

> ***Note:*** *In this lab you are using Ubuntu inside UTM (a virtual machine), so your disk will appear as vda (not sda). On a real physical Linux machine it would be sda. On macOS it is always disk0. Always check with lsblk -f or diskutil list first to confirm your exact device name before running commands.*

4.1 — Check File Type

Run the following commands to identify the type of different system files on Linux:

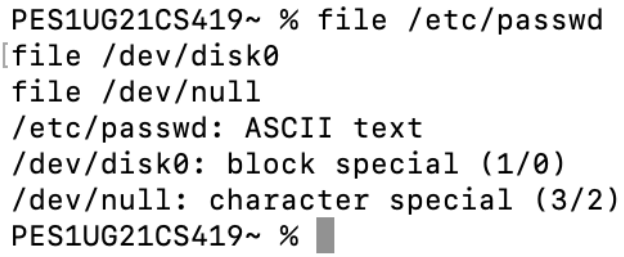
```
file /etc/passwd
file /dev/vda
file /dev/null
```



file command output on Linux showing file types

For macOS, use:

```
file /etc/passwd
file /dev/disk0
file /dev/null
```



file command output on macOS showing file types

What to notice:

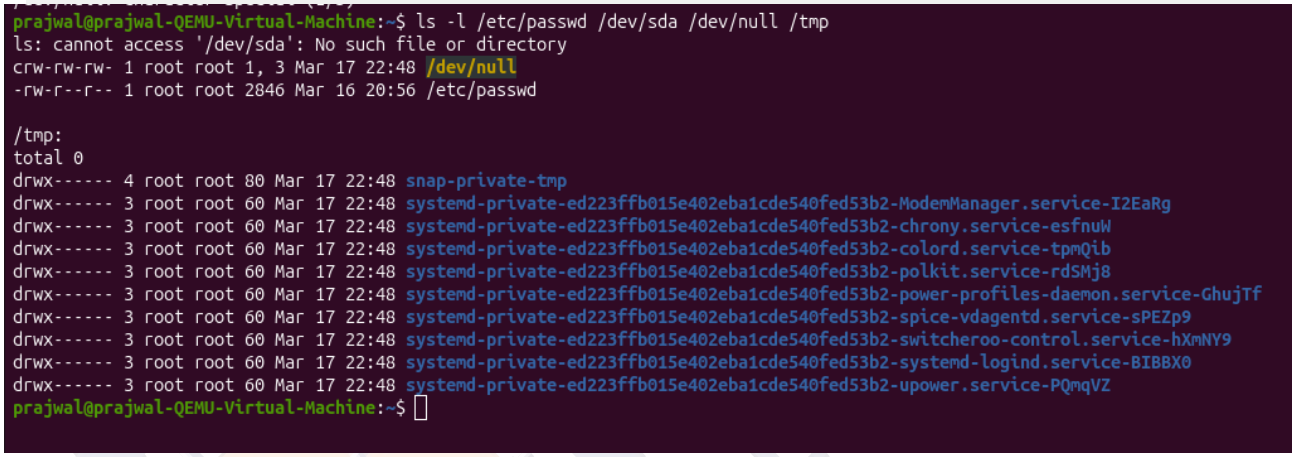
- **/etc/passwd:** is a regular text file.

- **/dev/sda / /dev/disk0:** is a block device (represents storage disk).
- **/dev/null:** is a character device (discards output).
- In Linux/macOS, everything is treated as a file.

4.2 — Identify File Type from Listing

Use ls -l to view file type indicators in the listing (Linux):

```
ls -l /etc/passwd /dev/vda /dev/null
```



ls -l output on Linux showing file type characters

For macOS:

```
ls -l /etc/passwd /dev/disk0 /dev/null
```

```
PES1UG21CS419~ % ls -l /etc/passwd /dev/disk0 /dev/null /tmp

brw-r-----  1 root  operator  0x1000000 12 Mar 11:06 /dev/disk0
crw-rw-rw-  1 root  wheel     0x3000002 17 Mar 22:58 /dev/null
-rw-r--r--  1 root  wheel          9344 25 Feb 09:11 /etc/passwd
lrwxr-xr-x@ 1 root  wheel            11 25 Feb 09:11 /tmp -> private/tmp
PES1UG21CS419~ %
```

ls -l output on macOS showing file type characters

What to notice — first character indicates file type:

Symbol	Meaning
-	Regular file
d	Directory
b	Block device

c	Character device
l	Symbolic link
p	Named pipe

4.3 — Read Inode Metadata of a File

The stat command shows detailed inode metadata for a file (Linux):

```
stat /etc/passwd
```

```
prajwal@prajwal-QEMU-Virtual-Machine:~$ stat /etc/passwd
  File: /etc/passwd
  size: 2846            Blocks: 8          IO Block: 4096   regular file
Device: fd01h/64769d    Inode: 135247      Links: 1
Access: (0644/-rw-r--r--)  Uid: (    0/    root)   Gid: (    0/    root)
Access: 2026-03-17 22:48:36.422000000 +0530
Modify: 2026-03-16 20:56:03.050104087 +0530
Change: 2026-03-16 20:56:03.050104087 +0530
 Birth: 2026-03-16 20:56:03.049072420 +0530
prajwal@prajwal-QEMU-Virtual-Machine:~$
```

stat command output on Linux showing inode metadata

For macOS:

```
stat /etc/passwd
```

```
PES1UG21CS419~ %
stat /etc/passwd

16777234 85144848 -rw-r--r-- 1 root wheel 0 9344 "Feb 25 09:11:32 2026" "Feb 25 09:11:32 2026" "Mar 11 22:09:08 2026" "Feb 25 09:11:32 2026" 4096 8 0x20 /etc/passwd
PES1UG21CS419~ %
```

stat command output on macOS showing inode metadata

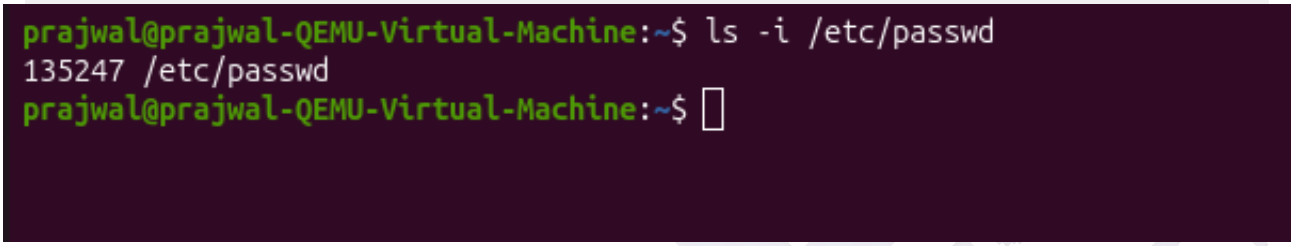
What to notice:

- **Inode:** unique identifier of the file on the filesystem.
- **Links:** number of references/hard links to the file.
- **Access (atime):** last time the file was read.
- **Modify (mtime):** last time the file content was changed.
- **Change (ctime):** last time the file metadata was changed.
- These timestamps are very important in forensic timeline analysis.

4.4 — Get Inode Number of a File

Use ls -i to get the inode number of a file (Linux):

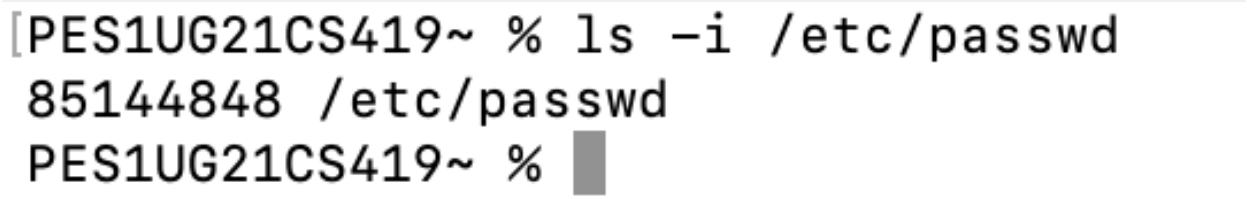
```
ls -i /etc/passwd
```



ls -i output showing inode number on Linux

For macOS:

```
ls -i /etc/passwd
```



ls -i output showing inode number on macOS

Part 5: macOS Forensic Commands

The following commands are executed in the macOS Terminal. They allow collection of forensic evidence from a live macOS system.

5.1 — Display Hardware Information

```
system_profiler SPHardwareDataType
```

```
PES1UG21CS419~ % system_profiler SPHardwareDataType
2026-03-16 22:10:12.054 system_profiler[80142:1983759] hw.cpufamily: 0xda33d83d
Hardware:

    Hardware Overview:

      Model Name: MacBook Air
      Model Identifier: Mac14,2
      Model Number: MLY33HN/A
      Chip: Apple M2
      Total Number of Cores: 8 (4 Performance and 4 Efficiency)
      Memory: 8 GB
      System Firmware Version: 13822.81.10
      OS Loader Version: 13822.81.10
      Serial Number (system): HJHH7CMY97
      Hardware UUID: 3FE06E8A-825B-5701-9979-9F8FD9995A3F
      Provisioning UDID: 00008112-001509081E8A201E
      Activation Lock Status: Enabled

PES1UG21CS419~ %
```

system_profiler output showing hardware details

Displays model identifier, serial number, processor type, and installed memory.

5.2 — Show Logged-in Users

```
who
```

```
PES1UG21CS419~ % who
apple            console      12 Mar 11:06
apple            ttys000      16 Mar 22:10
PES1UG21CS419~ %
```

who command output showing logged-in users

Lists all users currently logged in along with login time and terminal info.

5.3 — Display Running Processes

```
ps aux
```

```
PES1UG21CS419~ % ps aux
USER               PID  %CPU %MEM      VSZ    RSS   TT  STAT STARTED      TIME COMMAND
apple            78812   8.3  0.8 436166144  67488   ??  S     8:54PM   3:50.09 /Applications/UTM.app/Contents/MacOS/UTM
_windowserver      402   6.9  0.9 436499536  75520   ??  Ss   Thu11AM 324:15.06 /System/Library/PrivateFrameworks/SkyLight.framework/Resources/WindowServer -daemon
root               369   3.5  0.1 435347760   7696   ??  Ss   Thu11AM   3:30.73 /usr/libexec/opendirectoryd
apple            78837   3.0  1.0 440023744  80288   ??  S     8:55PM  13:44.85 /Applications/UTM.app/Contents/XPCServices/QEMUHelper.xpc/Contents/MacOS/QEMULauncher.app/Contents/MacOS/QEMULauncher /A
apple            80115   2.7  0.7 435877616  57280   ??  S    10:10PM   0:01.87 /System/Applications/Utilities/Terminal.app/Contents/MacOS/Terminal
```

ps aux output showing running processes

Snapshot of all running processes with CPU/memory usage and user accounts.

5.4 — Check Network Connections

```
netstat -an
```

```
PES1UG21CS419~ % netstat -an
Active Internet connections (including servers)
Proto Recv-Q Send-Q  Local Address          Foreign Address        (state)
tcp6       0      0  *.49880                *.*                    LISTEN
tcp4       0      0  *.49880                *.*                    LISTEN
tcp4       0      0  172.20.10.2.49772      74.125.24.188.5228     FIN_WAIT_1
tcp6       0      0  *.53                   *.*                    LISTEN
tcp4       0      0  *.53                   *.*                    LISTEN
tcp6       0      0  fe80::3cd8:f4fb:.1032  fe80::9daa:78f4:.1025  ESTABLISHED
tcp6       0      0  fe80::3cd8:f4fb:.1024  fe80::9daa:78f4:.1024  ESTABLISHED
tcp46      0      0  *.64479                *.*                    LISTEN
tcp4       0      0  127.0.0.1.8021         *.*                    LISTEN
tcp6       0      0  ::1.8021               *.*                    LISTEN
udp46      0      0  *.59239                *.*
udp46      0      0  *.547                  *.*
udp6       0      0  *.53                   *.*
udp4       0      0  *.53                   *.*
udp4       0      0  *.67                   *.*
udp46      0      0  *.5353                 *.*
udp46      0      0  *.5353                 *.*
udp46      0      0  *.5353                 *.*
udp46      0      0  *.5353                 *.*
udp46      0      0  *.5353                 *.*
udp46      0      0  *.5353                 *.*
udp46      0      0  *.5353                 *.*
```

netstat -an output showing network connections

Lists all active network connections and listening ports. Useful for spotting suspicious outbound connections.

5.5 — Display Disk Information

```
diskutil list
```

```
PES1UG21CS419~ % diskutil list
/dev/disk0 (internal, physical):
   #:                       TYPE NAME                    SIZE       IDENTIFIER
   0:      GUID_partition_scheme                        *251.0 GB   disk0
   1:             Apple_APFS_ISC Container disk1         524.3 MB   disk0s1
   2:                 Apple_APFS Container disk3         245.1 GB   disk0s2
   3:        Apple_APFS_Recovery Container disk2         5.4 GB     disk0s3
```

diskutil list output showing disk partitions

Lists all attached disks and their partition schemes.

5.6 — Read macOS Version (System Artifact)

```
cat /System/Library/CoreServices/SystemVersion.plist
```

```
PES1UG21CS419~ % cat /System/Library/CoreServices/SystemVersion.plist
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
        <key>BuildID</key>
        <string>E0AF3C06-11FB-11F1-A7CC-608CDE06C496</string>
        <key>ProductBuildVersion</key>
        <string>25D2128</string>
        <key>ProductCopyright</key>
        <string>1983-2026 Apple Inc.</string>
        <key>ProductName</key>
        <string>macOS</string>
        <key>ProductUserVisibleVersion</key>
        <string>26.3.1</string>
        <key>ProductVersion</key>
        <string>26.3.1</string>
        <key>iOSSupportVersion</key>
        <string>26.3</string>
</dict>
</plist>
PES1UG21CS419~ %
```

SystemVersion.plist content showing macOS version

Reads the system version plist to determine the exact macOS version. A critical artifact during forensic examination.

5.7 — Read plist File in Human-Readable Format

```
plutil -convert xml1 -o - ~/Library/Preferences/com.apple.finder.plist
```

```
PES1UG21CS419~ % plutil -convert xml1 -o - ~/Library/Preferences/com.apple.finder.plist
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/Propert
<plist version="1.0">
<dict>
        <key>ComputerViewSettings</key>
        <dict>
                <key>CustomViewStyleVersion</key>
                <integer>1</integer>
                <key>IconViewSettings</key>
                <dict>
                        <key>arrangeBy</key>
                        <string>name</string>
                        <key>backgroundColorBlue</key>
                        <real>1</real>
                        <key>backgroundColorGreen</key>
                        <real>1</real>
                        <key>backgroundColorRed</key>
                        <real>1</real>
                        <key>backgroundType</key>
                        <integer>0</integer>
                        <key>gridOffsetX</key>
                        <real>0.0</real>
```

plutil output of Finder preferences in XML format

Converts a binary .plist file to human-readable XML. macOS stores many user preferences and artifacts in plist format.

5.8 — View Safari Browsing History

Safari history is stored as an SQLite database. Query it using sqlite3:

```
sqlite3 ~/Library/Safari/History.db

.tables

SELECT * FROM history_items LIMIT 20;
```

```
PES1UG21CS419~ % sqlite3 ~/Library/Safari/History.db \
"SELECT datetime(visit_time+978307200,'unixepoch'), url
FROM history_visits
JOIN history_items ON history_visits.history_item = history_items.id
ORDER BY visit_time DESC LIMIT 5;"
2026-03-16 16:53:02|https://www.pesuacademy.com/Academy/
2026-03-16 16:53:02|http://www.pesuacademy.com/
2026-03-16 16:52:52|https://www.google.com/
2026-03-16 16:52:52|https://www.google.com/
2026-03-16 16:52:52|http://www.google.com/
PES1UG21CS419~ %
```

Safari browsing history queried via sqlite3

What to notice:

- **Storage format:** Safari history is stored as an SQLite database.
- **history_items table:** contains visited URLs and visit counts.
- **Forensic value:** browser history is a critical artifact in investigations.